

# Agent Engine 上的有狀態資料科學代理

來源：<https://codelabs.developers.google.com/next26/adk-deploy-scale?hl=zh-tw>

步驟數：10

建立有狀態的資料科學代理程式，可查詢 BigQuery、記住跨工作階段的使用者偏好設定，並透過 Cloud Trace 提供可觀測性。將其部署至 Agent Engine。

---

# 目錄

1. [總覽](#)
2. [設定環境](#)
3. [建立代理程式](#)
4. [部署至 Agent Engine](#)
5. [授予 BigQuery 權限](#)
6. [測試已部署的代理](#)
7. [測試記憶體持續性](#)
8. [認識 Observability](#)
9. [清除](#)
10. [恭喜](#)

步驟 1 · 約 2 分鐘

## 1. 總覽

在本程式碼研究室中，您將建構資料科學代理程式，從 BigQuery 公開資料集查詢實際資料，並在不同工作階段中記住您的偏好設定。接著，您會將其部署至 Agent Engine，這項全代管的 Google Cloud 服務會處理基礎架構、資源調度和工作階段管理。

代理程式會逐步啟用三項核心功能：

- **BigQuery 工具集**：代理程式會探索結構定義，並對實際的 BigQuery 資料集執行 SQL 查詢，無論是在本機或部署時都適用。
- **Memory Bank**：部署後，代理程式會記住使用者偏好設定和對話脈絡，即使工作階段中斷也不例外。
- **可觀測性**：Cloud Trace 會透過 OpenTelemetry 檢測，擷取代理程式的推論步驟、工具呼叫和延遲時間。

## 課程內容

- 如何使用 `BigQueryToolset` 建立 ADK 代理，存取真實資料
- 如何設定 Memory Bank，確保跨工作階段的持續性
- 如何使用 `adk deploy` 將代理部署至 Agent Engine
- 如何授予已部署代理程式的服務帳戶 IAM 權限
- 如何測試記憶體持續性和可觀測性

## 軟硬體需求

- 已啟用計費功能的 Google Cloud 雲端專案
- [Google Cloud SDK](#) ( `gcloud` CLI)
- 網路瀏覽器，例如 [Chrome](#)
- [uv](#) (Python 套件管理員)
- Python 3.12 以上版本 (如有需要，`uv` 會自動安裝)

ADK (Agent Development Kit) 是 Google 用於建構 AI 代理的架構。本程式碼實驗室會使用 ADK 建立代理，並將其部署至 Agent Engine。

本程式碼研究室適合對 Python 和 Google Cloud 有基本認識的中階開發人員。

完成本程式碼研究室大約需要 30 分鐘 (包括部署作業的 5 到 10 分鐘)。

本程式碼研究室建立的資源費用應低於 \$5 美元。

---

步驟 2 · 約 5 分鐘

## 2. 設定環境

### 從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

info

### 從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

啟用

### 建立 Google Cloud 專案

1. 在 [Google Cloud 控制台](#) 的專案選取器頁面中，[選取或建立 Google Cloud 專案](#)。
2. 確認 Cloud 專案已啟用計費功能。瞭解如何[檢查專案是否已啟用計費功能](#)。

## 設定環境變數

在您建立的 GCP 專案中開啟 [Cloud Shell 編輯器](#)。

然後建立「終端機」>「新終端機」，並執行下列指令。

```
export GOOGLE_CLOUD_PROJECT=<INSERT_YOUR_GCP_PROJECT_HERE>
export GOOGLE_CLOUD_LOCATION=us-central1
export GOOGLE_GENAI_USE_VERTEXAI=True
```

## 啟用 API

在終端機中執行下列指令。

```
gcloud services enable \
  aiplatform.googleapis.com \
  bigquery.googleapis.com \
  telemetry.googleapis.com \
  --project=$GOOGLE_CLOUD_PROJECT
```

- **AI Platform API** ( [aiplatform.googleapis.com](#) ) - Agent Engine 主機
- **BigQuery API** ( [bigquery.googleapis.com](#) ) - 針對公開和私人資料集執行 SQL 查詢
- **Telemetry API** ( [telemetry.googleapis.com](#) ) - OpenTelemetry 追蹤記錄，用於代理程式觀測能力

API 啟用程序最多需要一分鐘才能完成。如果在後續步驟中看到「API not enabled」(API 未啟用) 錯誤，請稍候片刻再重試。

## 建立虛擬環境並安裝 ADK

```
uv venv .venv --python 3.12
source .venv/bin/activate
uv pip install google-adk google-auth
```

`google-adk` 套件包含 `adk` CLI 工具，可用於測試及部署代理程式。

---

步驟 3 · 約 5 分鐘

### 3. 建立代理程式

建立新的代理程式專案目錄。後續所有指令都應從這個工作目錄 (`data_science_agent/` 的上層目錄) 執行：

```
mkdir data_science_agent
```

最終的目錄結構如下所示：

```
./
data_science_agent/
  __init__.py
  agent.py
  requirements.txt    # created in the Deploy step
  .env                # created in the Deploy step
```

您現在要建立 `__init__.py` 和 `agent.py`，然後在「Deploy」(部署) 步驟中新增 `requirements.txt` 和 `.env`。

建立 `data_science_agent/__init__.py`，ADK 必須有這個檔案才能探索及載入代理：

```
from . import agent # noqa: F401 -- required by `adk eval` and `adk web`
```

建立 `data_science_agent/agent.py`：

這個代理程式會連線至 BigQuery 擷取資料，並將工作階段保留在 Memory Bank 中。

記憶體會在部署時自動啟用，`GOOGLE_CLOUD_AGENT_ENGINE_ID` 環境變數是由 Agent Engine 執行階段設定，在本機執行時則不會出現。

```

from __future__ import annotations

import os

from google.adk.agents import LlmAgent
from google.adk.agents.callback_context import CallbackContext
from google.adk.apps import App
from google.adk.tools.bigquery import BigQueryCredentialsConfig
from google.adk.tools.bigquery import BigQueryToolset
from google.adk.tools.preload_memory_tool import PreloadMemoryTool
import google.auth

PROJECT_ID = os.getenv("GOOGLE_CLOUD_PROJECT")
if not PROJECT_ID:
    raise ValueError(
        "GOOGLE_CLOUD_PROJECT environment variable is required. "
        "Set it with: export GOOGLE_CLOUD_PROJECT=<your-project-id>"
    )

credentials, _ = google.auth.default()
bq_toolset =
BigQueryToolset(credentials_config=BigQueryCredentialsConfig(credentials=credentials))

# GOOGLE_CLOUD_AGENT_ENGINE_ID is set automatically by the Agent Engine runtime.
agent_engine_id = os.getenv("GOOGLE_CLOUD_AGENT_ENGINE_ID")

async def _save_memory(callback_context: CallbackContext) -> None:
    """Persist the session to Memory Bank after each agent run.

    Only activates on Agent Engine where Memory Bank is available.
    """
    if agent_engine_id:
        await callback_context.add_session_to_memory()

root_agent = LlmAgent(
    name="data_science_agent",
    model="gemini-2.5-pro",
    instruction=(
        "You are an expert Data Science Agent. "
        "Your goal is to query enterprise BigQuery datasets, analyze the data, "
        "and summarize your findings. "
        f"When executing SQL queries, use project_id `{PROJECT_ID}` as the "
        "billing project unless the user specifies a different one. "
        "Present results clearly with formatted numbers. "
        "Remember user preferences like preferred regions, date ranges, "
        "or analysis formats across conversations."
    ),
    tools=[bq_toolset, PreloadMemoryTool()],
    after_agent_callback=_save_memory,
)

```

```
app = App(  
    name="data_science_agent",  
    root_agent=root_agent,  
)
```

讓我們逐步瞭解這段程式碼的作用：

1. **BigQueryToolset** 為代理程式提供 `execute_sql`、`list_table_ids` 和 `get_table_info` 等工具，可探索架構並查詢呼叫端有權存取的任何資料集。
  2. **PreloadMemoryTool** 會在每次呼叫 LLM 前，搜尋 Memory Bank 中與使用者訊息相關的內容，自動擷取相關記憶。每次執行代理後，`_save_memory` 回呼都會將工作階段保留在 Memory Bank 中，因此代理可以在日後的工作階段中回想脈絡。
  3. **App** 會將根代理程式包裝成可部署的應用程式，供 Agent Engine 提供服務。`name` 必須與目錄名稱 (`data_science_agent`) 相符，`adk web` 會使用這個名稱尋找及載入代理程式。
  4. **指令**會告知代理程式使用報帳專案進行 SQL 查詢，並記住使用者偏好設定。
-



## 4. 部署至 Agent Engine

在 `data_science_agent` 目錄中建立 `requirements.txt` 檔案：

```
google-adk>=1.26.0
google-genai>=1.27.0
google-auth>=2.0.0
python-dotenv>=1.1.0
opentelemetry-instrumentation-fastapi
opentelemetry-instrumentation-google-genai
opentelemetry-instrumentation-httpx
opentelemetry-instrumentation-grpc
```

- `google-adk` 和 `google-genai`：ADK 架構和 Gemini 用戶端
- `google-auth` - Google Cloud 驗證
- `python-dotenv`：在啟動時載入 `.env` 檔案
- 這四個 `opentelemetry-instrumentation-*` 套件可啟用稍後會介紹的可觀測性功能。這些工具會檢測 FastAPI HTTP 要求、Gemini 模型呼叫和內部 gRPC/HTTP 通訊，以便在 Agent Engine 的「追蹤」分頁中顯示追蹤記錄。

請「不要」在 `requirements.txt` 中新增 `google-cloud-aiplatform[agent-engines]`。ADK 部署 CLI 會在部署期間自動新增這項依附元件。使用不同版本重新宣告可能會導致 OpenTelemetry 套件版本衝突，進而默默中斷檢測功能，導致追蹤記錄不會顯示在「追蹤記錄」分頁中。

在 `data_science_agent` 目錄中建立 `.env` 檔案，以啟用已部署代理程式的遙測功能：

```
GOOGLE_CLOUD_AGENT_ENGINE_ENABLE_TELEMETRY=true
OTEL_INSTRUMENTATION_GENAI_CAPTURE_MESSAGE_CONTENT=true
```

- `GOOGLE_CLOUD_AGENT_ENGINE_ENABLE_TELEMETRY`：在 Agent Engine 執行階段啟用 OpenTelemetry 管道。
- `OTEL_INSTRUMENTATION_GENAI_CAPTURE_MESSAGE_CONTENT`：記錄完整的提示詞輸入內容和代理回覆，有助於偵錯。

`OTEL_INSTRUMENTATION_GENAI_CAPTURE_MESSAGE_CONTENT` 設定會將使用者提示詞和代理程式回覆的完整內容記錄到 Cloud Logging。這項功能有助於偵錯，但可能會擷取敏感資料或個人識別資訊。在正式環境中，請將此值設為 `false`，或確保您已制定適當的資料處理政策。

部署代理程式。最後一個引數 `data_science_agent` 是包含代理程式碼的目錄：

```
adk deploy agent_engine \  
  --project=$GOOGLE_CLOUD_PROJECT \  
  --region=$GOOGLE_CLOUD_LOCATION \  
  --display_name="Data Science Agent" \  
  --trace_to_cloud \  
  --otel_to_cloud \  
  data_science_agent
```

| 檢舉                                | 目的                         |
|-----------------------------------|----------------------------|
| <code>--project / --region</code> | 目標 Google Cloud 雲端專案和區域    |
| <code>--display_name</code>       | Cloud 控制台中顯示的人類可讀名稱        |
| <code>--trace_to_cloud</code>     | 為代理程式範圍啟用 Cloud Trace 匯出工具 |
| <code>--otel_to_cloud</code>      | 啟用 OpenTelemetry 檢測管道      |

部署作業通常會在 **5 到 10 分鐘**內完成。完成後，您會看到：`Created agent engine:`  
`projects/.../reasoningEngines/`

等待期間，請參閱[Memory Bank 文件](#)，進一步瞭解跨工作階段記憶功能。

`adk deploy` 列印的網址可能不正確。如果連結無法使用，請前往 Cloud 控制台的「Agent Engine」頁面，然後按一下您的代理程式。

部署至 Agent Engine 時，系統會自動啟用兩項功能：

- **Memory Bank**：`PreloadMemoryTool` 連結至 Agent Engine Memory Bank，並 `_save_memory` 自動保留工作階段。
- **可觀測性**：Cloud Trace 會擷取代理程式的推論步驟、工具呼叫和延遲時間。

## 5. 授予 BigQuery 權限

您必須授予 Agent Engine 服務帳戶 BigQuery 存取權。部署後，代理程式會以 Google 代管的服務帳戶 (而非您的個人憑證) 執行，因此需要明確的權限才能執行 SQL 查詢。

```
PROJECT_NUMBER=$(gcloud projects describe $GOOGLE_CLOUD_PROJECT \
  --format='value(projectNumber)')

SA="service-${PROJECT_NUMBER}@gcp-sa-aiplatform-re.iam.gserviceaccount.com"

# Required to execute SQL queries
gcloud projects add-iam-policy-binding $GOOGLE_CLOUD_PROJECT \
  --member="serviceAccount:${SA}" \
  --role="roles/bigquery.jobUser"

# Required to read table metadata and data
gcloud projects add-iam-policy-binding $GOOGLE_CLOUD_PROJECT \
  --member="serviceAccount:${SA}" \
  --role="roles/bigquery.dataViewer"
```

每個指令成功執行後，都會輸出 `Updated IAM policy for project [...]`。

如果略過這個步驟，部署的代理程式在執行 SQL 查詢時會傳回 **403 錯誤**。您需要**報帳專案** (您的專案) 的 `bigquery.jobUser` 角色，而非資料專案。

注意：`bigquery.dataViewer` 角色可讀取專案中的所有資料集。如要在正式環境中使用，建議改為在資料集層級授予這個角色。

## 6. 測試已部署的代理

在 Google Cloud 控制台中開啟 [Agent Engine 頁面](#)。按一下已部署的代理程式，開啟 **Agent Engine Playground**。

代理程式會進行推理、呼叫工具並傳回結果，因此每項查詢可能需要 10 到 30 秒才能完成。

測試 BigQuery 功能：

- 「List the tables in bigquery-public-data.hacker\_news」
    - 預期行為：**代理程式會呼叫 `list_table_ids`，並傳回包含 `full` 的資料表名稱。
  - 「Find the number of posts per year in bigquery-public-data.hacker\_news.full」(在 bigquery-public-data.hacker\_news.full 中找出每年發布的貼文數量)
    - 預期：**代理程式會使用 SQL 查詢呼叫 `execute_sql`，並傳回年份和貼文計數的表格。
  - 「貼文的年增率是多少？」
    - 預期：**代理程式會使用計算百分比變更的 SQL 查詢呼叫 `execute_sql`，並傳回結果。
-

## 7. 測試記憶體持續性

在 Playground 中，教導代理程式偏好設定：

1. 「記住我最愛的資料集是 `bigquery-public-data.hacker_news`」
2. 「有哪些表格？」

等待幾秒鐘，讓記憶體持續存在 ( `_save_memory` 回呼會在代理程式回應後執行)。

現在請點選 Playground 側欄中的「+ New Session」按鈕啟動新工作階段，然後提出以下問題：

3. 「我最喜歡的資料集是什麼？」

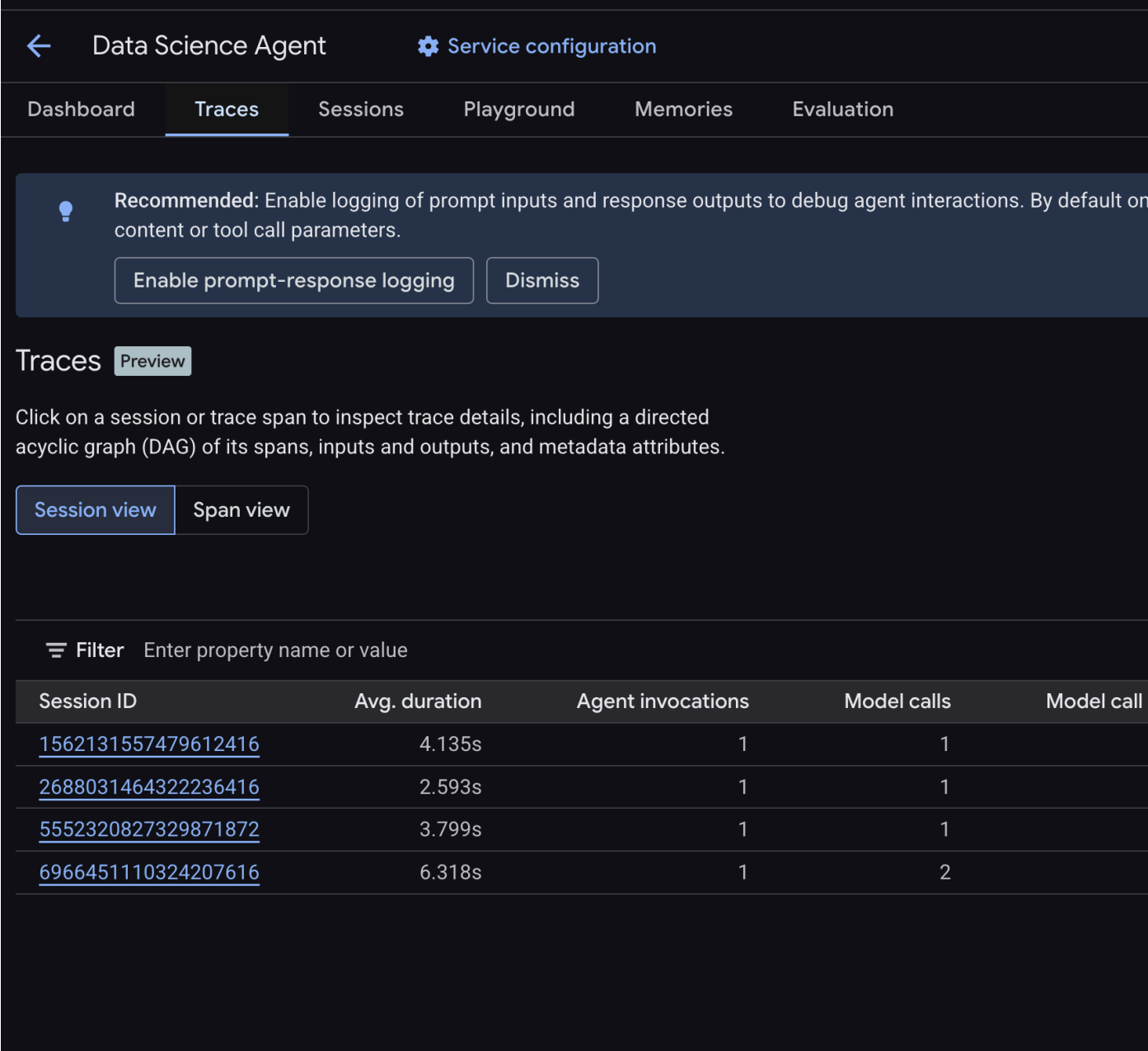
即使這是沒有對話記錄的全新工作階段，代理程式也應回想 `bigquery-public-data.hacker_news`。運作原理：

- `_save_memory` 透過 `callback_context.add_session_to_memory()` 將每個工作階段的內容儲存至 Memory Bank
- `PreloadMemoryTool` 在每次呼叫 LLM 前擷取相關記憶內容
- Memory Bank 會比對內容的語意，而不只是關鍵字

如果服務專員沒有記住你的偏好設定，請等待 10 秒，然後在另一個新工作階段中重試。記憶體持續性是非同步作業，可能需要一段時間才能完成。

## 8. 認識 Observability

在 Cloud 控制台中，前往已部署的代理程式，然後按一下「追蹤」分頁。



您應該會看到「工作階段資料表」，列出您在先前步驟中執行的測試查詢工作階段。表格會顯示每個工作階段的摘要指標，包括平均時間、模型呼叫、工具呼叫、權杖用量和任何錯誤。

點選工作階段即可查看追蹤記錄詳細資料，包括：

- 代理程式跨度的有向無環圖 (DAG)，顯示代理程式推論、工具呼叫 (BigQuery 查詢) 和延遲的逐步細目
- 每個範圍的輸入和輸出 (透過 `.env` 中的 `OTEL_INSTRUMENTATION_GENAI_CAPTURE_MESSAGE_CONTENT` 環境變數啟用)
- 中繼資料屬性，例如範圍 ID、追蹤 ID 和時間

您也可以切換至「範圍檢視畫面」(頂端的切換按鈕)，查看所有工作階段的個別範圍。

查詢完成後，追蹤記錄可能需要 1 到 2 分鐘才會顯示。如果尚未看到任何追蹤記錄，請稍候片刻並重新整理頁面。

## 追蹤功能的運作方式

使用 `--trace_to_cloud` 和 `--otel_to_cloud` 部署時，Agent Engine 執行階段會初始化 OpenTelemetry 管道，該管道會執行下列操作：

1. 建立 **TracerProvider**，其中包含 OTLP 匯出工具，可將時距傳送至 `telemetry.googleapis.com`
2. 使用 `requirements.txt` 中的四個**檢測套件**，從主要程式庫 (FastAPI、Gemini、httpx、gRPC) 擷取範圍 - `google-genai` 由執行階段明確檢測，其他則透過 OpenTelemetry 自動探索功能提供
3. 將批次和匯出範圍傳送至 Telemetry API，由「追蹤」分頁讀取

Agent Engine 基本映像檔提供 OpenTelemetry SDK 和匯出工具，但**不包含檢測套件**。因此，您的 `requirements.txt` 必須列出所有四個項目，否則系統不會建立任何範圍，也不會顯示任何追蹤記錄。

## 疑難排解

如果幾分鐘後仍未顯示追蹤記錄，請按照下列步驟操作：

1. **確認已啟用 Telemetry API**：您在設定步驟中已啟用這項 API。驗證方式：`gcloud services list --enabled --project=$GOOGLE_CLOUD_PROJECT | grep telemetry`
2. **檢查 Cloud Logging 是否有警告**：依序前往「Logging」>「Logs Explorer」，然後搜尋 `"telemetry enabled but proceeding without"`。如果看到有關生成式 AI 插碼的警告，表示 `requirements.txt` 中缺少 `opentelemetry-instrumentation-google-genai`。
3. 請勿在 `requirements.txt` 中加入 `google-cloud-aiplatform[agent-engines]`。ADK 部署 CLI 會自動新增此項目；使用不同版本重新宣告可能會導致 OpenTelemetry 套件衝突，並在不發出通知的情況下中斷檢測。

## 9. 清除

如要避免持續產生費用，請刪除在本程式碼研究室中建立的資源。

在 Cloud 控制台的「Agent Engine」頁面**刪除已部署的代理程式**。選取代理程式，然後按一下「刪除」。

本程式碼研究室中的 BigQuery 查詢使用公開資料集，因此不會產生儲存空間費用。您授予的 IAM 角色仍會保留在專案中，但不會產生任何費用。

如果您是特地為這個程式碼研究室建立專案，可以改為刪除整個專案：

```
gcloud projects delete ${GOOGLE_CLOUD_PROJECT}
```

(選用) 清理本機環境：

```
deactivate  
rm -rf .venv data_science_agent
```

---



## 10. 恭喜

您已建構有狀態的資料科學家代理程式，並將其部署至 Agent Engine！

### 目前所學內容

- 如何使用 `BigQueryToolset` 建立 ADK 代理，存取真實資料
- 如何使用 `PreloadMemoryTool` 和 `after_agent_callback`，透過 Memory Bank 啟用永久記憶體
- 如何授予已部署代理程式的服務帳戶 IAM 權限
- 如何部署至 Agent Engine，並透過 Cloud Trace 啟用可觀測性

### 後續步驟

- 授予 Agent Engine 服務帳戶資料存取權，即可查詢自己的私人 BigQuery 資料集
- 新增「程式碼執行」，在安全沙箱中執行 Python 分析
- 設定 [Cloud Trace 可觀測性](#) 資訊主頁，監控正式環境中的代理程式
- 使用 [MCP 工具](#) 將結果發布至 Google Workspace

### 參考文件

- [ADK 說明文件](#)
  - [Agent Engine 說明文件](#)
  - [Memory Bank 說明文件](#)
  - [Agent Engine 部署指南](#)
-